

```

const rl = @import("raylib");
const std = @import("std");
const print = std.debug.print;

const sqlite3 = @import("lib/Database.zig");
const cardboard = @import("lib/Cardboard.zig");
const gamemap = @import("lib/GameMap.zig");
const tile = @import("lib/Tile.zig");

pub fn main() anyerror!void {
    const db = sqlite3.Database.init();
    defer db.close();
    db.foo();

    var arena = std.heap.ArenaAllocator.init(std.heap.page_allocator);
    defer arena.deinit();
    const allocator = arena.allocator();

    var Assets = std.ArrayList(rl.Image).empty;
    defer Assets.deinit(allocator);

    var Tiles = std.ArrayList(tile.Tile).empty;
    defer Tiles.deinit(allocator);

    // *****
    const screenWidth = 1280;
    const screenHeight = 720;

    rl.setConfigFlags({ .window_undecorated = true });
    rl.initWindow(screenWidth, screenHeight, "The Long Road");
    defer rl.closeWindow();

    // =====
    try db.add_map_tiles(allocator, &Assets, &Tiles);
    print("tile count: {d}\n", .{Tiles.items.len});

    const tileLetters = try db.get_tileLetters(allocator);
    defer allocator.free(tileLetters);

    var gMap = gamemap.GameMap.init(tileLetters);

    for (0..4) |row| {
        for (0..4) |col| {
            print("{d},{d}", .{ col, row });
            print("{d}, ", .{gMap.GMap[@intCast(col)][@intCast(row)]]);
        }
        print("\n", .{});
    }

    print("{d}\n", .{Tiles.items.len});
    for (0..Tiles.items.len) |x| {
        print("{d}:{d}, ", .{ Tiles.items[x].id, Tiles.items[x].rotation });
    }
    print("\n", .{});

    gMap.modify_GMap(&Tiles);

    for (0..4) |row| {
        for (0..4) |col| {
            print("{d},{d}", .{ col, row });
            print("{d}, ", .{gMap.GMap[@intCast(col)][@intCast(row)]]);
        }
        print("\n", .{});
    }
    print("\n", .{});

    for (0..4) |row| {
        for (0..4) |col| {
            if (gMap.GMap[@intCast(col)][@intCast(row)] >= 0) {
                print("{d},{d}", .{ col, row });
                print("{d}, ", .{gMap.GMap[@intCast(col)][@intCast(row)]]);
            }
        }
        print("\n", .{});
    }
    print("\n", .{});

    // =====
    const Unit_1f = cardboard.Cardboard.init(allocator, &Assets, "TLR/7th-Hamilton-F.png", 12);

```

```

const Unit_1b = cardboard.Cardboard.init(allocator, &Assets, "TLR/7th-Hamilton-B.png", 13);

var mapTile: rl.Texture = undefined;
mapTile = try rl.loadTextureFromImage(Assets.items.ptr[Tiles.items.ptr[0].index]);
var unit_1: rl.Texture = undefined;
unit_1 = try rl.loadTextureFromImage(Assets.items.ptr[Unit_1f.index]);

// =====
rl.setTargetFPS(12);

var camera = rl.Camera2D{
    .target = {.x = 0, .y = 0 },
    .offset = {.x = 0, .y = 0 },
    .zoom = 0.5,
    .rotation = 0,
};

while (!rl.windowShouldClose()) {
    // Translate based on mouse right click
    if (rl.isMouseButtonDown(.right)) {
        var delta = rl.getMouseDelta();
        delta = rl.math.vector2Scale(delta, -1.0 / camera.zoom);
        camera.target = rl.math.vector2Add(camera.target, delta);
    }

    // Zoom based on mouse wheel
    const wheel = rl.getMouseWheelMove();
    if (wheel != 0) {
        const mouseWorldPos = rl.getScreenToWorld2D(rl.getMousePosition(), camera);
        camera.offset = rl.getMousePosition();
        camera.target = mouseWorldPos;

        // Zoom increment
        var scaleFactor = 1.0 + (0.25 * @abs(wheel));
        if (wheel < 0) {
            scaleFactor = 1.0 / scaleFactor;
        }
        camera.zoom = rl.math.clamp(camera.zoom * scaleFactor, 0.125, 64.0);
    }

    // =====
    rl.beginDrawing();
    defer rl.endDrawing();

    rl.clearBackground(.black);

    {
        camera.begin();
        defer camera.end();

        // rl.clearBackground(.white);
        rl.drawTexture(mapTile, 0, 0, .white);
        rl.drawTexture(unit_1, 50, 50, .white);

        // Control frames speed
        if (rl.isKeyPressed(.right)) {
            mapTile = try rl.loadTextureFromImage(Assets.items.ptr[Tiles.items.ptr[0].index]);
            unit_1 = try rl.loadTextureFromImage(Assets.items.ptr[Unit_1f.index]);
        } else if (rl.isKeyPressed(.left)) {
            mapTile = try rl.loadTextureFromImage(Assets.items.ptr[Tiles.items.ptr[1].index]);
            unit_1 = try rl.loadTextureFromImage(Assets.items.ptr[Unit_1b.index]);
        }

        rl.drawText("Congrats! You created your first window!", 190, 200, 20, .black);
    }
}
}

```